

Testing pyruleanalyzer models on real Arduino and ESP32 boards

A guide to `examples/arduino_hardware_test.py` — from training in Python to validation on the physical board

1. What this script does

pyruleanalyzer already exports a trained model (Decision Tree, Random Forest or GBDT) to a self-contained Arduino sketch, with the trees turned into const arrays and a `pyra_predict()` function in C. What was missing was the answer to the next question: **does the board really classify the same way Python does?**

The script `examples/arduino_hardware_test.py` answers that with a *hardware-in-the-loop* (HIL) test: it trains the model, generates a test sketch, compiles it, flashes the board, sends real samples over the serial port and compares, one by one, the class returned by the microcontroller against the class predicted in Python — while also measuring the latency of each inference and the real Flash and SRAM usage.

The **production** sketch reads sensors inside `read_features()`; that is why it cannot be tested on its own — the data comes from the physical world. The trick is to invert the flow: the PC becomes the data source, and the **test** sketch reuses exactly the same arrays and the same `pyra_predict()`, replacing only the sensor reading with a serial protocol. What gets validated is therefore the very code that ships to the field.

How the test works: the PC becomes the data source

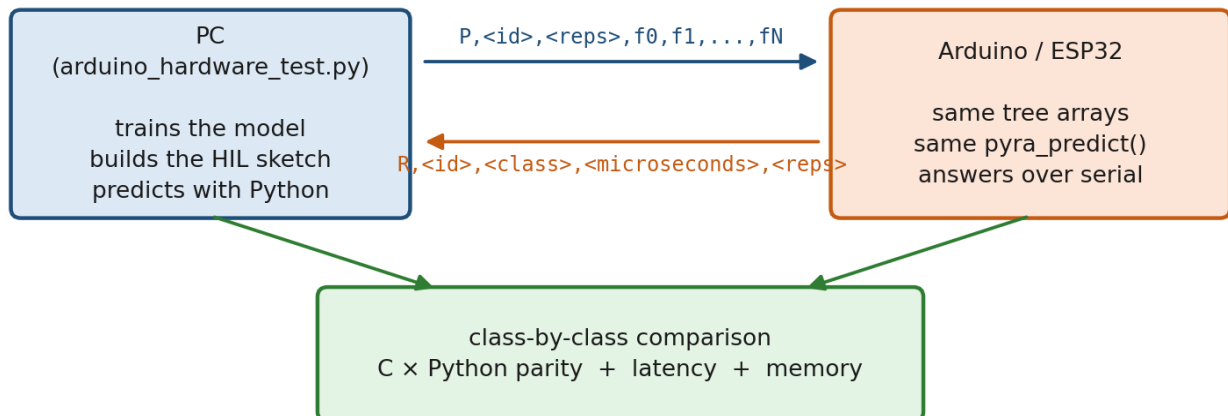


Figure 1 — the PC sends the feature vector; the board returns the class and the inference time. All that is left is comparing them.

2. Before you start: prerequisites

Install once, on the machine that will run the test:

```
pip install pyserial          # serial communication with the board

# arduino-cli (compile and flash) - https://arduino.github.io/arduino-cli/
winget install ArduinoSA.CLI  # Windows
```

```

arduino-cli core update-index
arduino-cli core install arduino:avr # Uno / Nano / Mega / Leonardo
arduino-cli core install esp32:esp32 # ESP32 (needs the board manager URL)

arduino-cli board list          # confirm the board shows up

```

- **Windows:** Nano and ESP32 clones need the CH340 or CP2102 driver.
- **g++** (optional but recommended): enables stage 3, which validates the C code on the PC with no board at all.
- **Close the Arduino IDE Serial Monitor** before running: the port is exclusive.

3. The test pipeline, stage by stage

The 7 stages of the pipeline

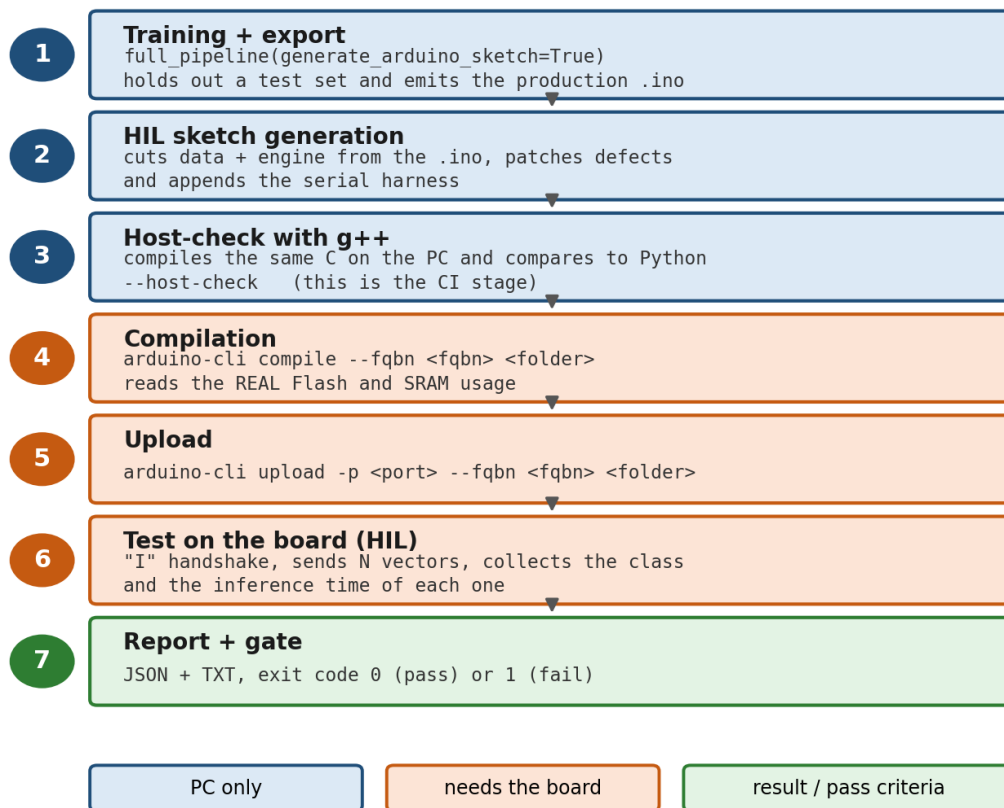


Figure 2 — the blue stages run on the PC alone; the orange ones need the board attached.

Stage 1 — Training and export

The script loads the dataset (a scikit-learn built-in or your own CSV), holds out a test set that does **not** take part in training, and calls `full_pipeline(generate_arduino_sketch=True)`. Out of it come the production `.ino` and the memory estimate (`memory_check`).

Stage 2 — HIL sketch generation

The script cuts from the `.ino` everything before the `ARDUINO SKETCH SECTION` marker — that is, the tree data and the inference engine — and appends the serial *harness* on top. The result lands in `files/hil/<name>_hil/<name>_hil.ino` (Arduino requires the folder to carry the same name as the file).

Along the way, this stage fixes two defects of the generator: (1) the `Feature` order: block is emitted without `//` and does not compile; (2) the production sketch declares `float features[]` and calls `pyra_predict((const double*)features)` — an invalid cast that reads garbage on any platform where `float != double`. The harness declares the buffer as `double` and uses no cast.

Stage 3 — Host-check with g++ (no hardware)

Before spending time flashing, the same C code is compiled on the PC with g++ and fed the test vectors. If $C \times \text{Python}$ parity already fails here, the problem is in the model or the exporter — not on the board. This is the stage you run in continuous integration.

Stage 4 — Compiling for the board

`arduino-cli compile` reports the **real** memory usage; the script extracts the `Sketch` uses ... bytes and `Global variables` use ... bytes lines and puts them in the report next to the estimate. The real number being larger than the estimate is expected — the Arduino runtime is included. What matters is the final percentage.

Stage 5 — Upload

Flashing via `arduino-cli upload -p <port>`. If the board already runs the HIL sketch, use `--skip-upload`; to skip compiling as well, `--skip-compile`.

Stage 6 — The test on the board

The script opens the serial port (opening it resets AVR boards, hence the `--boot-delay` wait), handshakes with the `I` command and then sends the samples. Each response carries the class and the total time of `--reps` inferences, so that the resolution of `micros()` does not dominate the measurement.

Stage 7 — Report and pass criteria

A `.json` and a `.txt` are written to `--report-dir`. The process exits with **code 0** (pass) or **1** (fail), ready to be used as a CI gate. It fails when: parity drops below `--min-parity` (default 100%), any timeout or invalid response shows up, the host-check fails, or Flash/SRAM exceed the limits of the board.

4. How to run it

4.1 With no board at all (start here)

Validates the whole pipeline up to stage 3. This is the command for your first run and for CI:

```
python examples/arduino_hardware_test.py --dataset iris \
    --host-check --no-hardware
```

4.2 Full cycle on a real board

```
# Arduino Uno on COM5 (Linux/Mac: /dev/ttyUSB0 or /dev/ttyACM0)
python examples/arduino_hardware_test.py --dataset iris \
    --board uno --port COM5 --host-check
```

If you omit `--port`, the script tries to find the board itself with `arduino-cli board list`.

4.3 Other useful scenarios

```
# ESP32 with Random Forest, 200 samples and 50 repetitions per sample
python examples/arduino_hardware_test.py --dataset wine \
    --model "Random Forest" --n-estimators 10 --max-depth 6 \
    --board esp32 --port COM7 --n-samples 200 --reps 50
```

```
# Your own CSV, with the board already flashed
python examples/arduino_hardware_test.py --csv files/dataset_iris.csv \
    --target Target --board nano --port COM3 --skip-upload

# Reusing an already trained model
python examples/arduino_hardware_test.py --model-pkl files/final_model.pkl \
    --board mega --port COM4
```

4.4 Main options

Option	What it does
--dataset / --csv	Built-in dataset (iris, wine, breast_cancer, digits) or your own CSV, with --target naming the label column.
--model	"Decision Tree" (default), "Random Forest" or "Gradient Boosting Decision Trees".
--n-estimators --max-depth	Model size. On Uno/Nano, keep the depth low and the number of trees small.
--board / --fqbn	uno, nano, nano_old, mega, leonardo, esp32 — or an explicit FQBN.
--port / --baud	Serial port and speed (default 115200). Without --port, it tries to auto-detect.
--n-samples / --reps	How many samples to send and how many repetitions per sample when measuring latency.
--host-check	Enables stage 3 (compiles the engine with g++ and compares against Python).
--no-hardware	Stops after stage 3: no compiling, no flashing.
--skip-compile --skip-upload	Skips compiling and/or flashing when the board is already set up.
--min-parity	Minimum parity for the test to pass (default 100%).
--output-dir / --report-dir	Where the generated artifacts and the reports go.

5. The serial protocol

Plain text, one message per line, terminated by `\n`. You can talk to the board by hand through the IDE Serial Monitor — handy for debugging:

Direction	Message	Meaning
PC -> board	I	Asks for the model metadata.
board -> PC	I,<algorithm>,<n_features>, <n_classes>,<n_trees>	Handshake response.
PC -> board	P,<id>,<reps>,<f0>,<f1>,...	Asks for the classification of one vector, repeated <reps> times.
board -> PC	R,<id>,<class>, <microseconds>,<reps>	Predicted class and total time of the repetitions.
board -> PC	E,<reason>	Parse error or unknown command.

A manual example, with a 4-feature model: sending `P, 0, 1, 5.1, 3.5, 1.4, 0.2` should return something like `R, 0, 0, 180, 1` — class 0, in 180 microseconds.

6. Reading the report

The generated `.txt` looks like this:

```

HIL REPORT - pyruleanalyzer on hardware
Model          : Decision Tree (Decision Tree)
Board / FQBN    : uno / arduino:avr:uno
Samples        : 50 (answered: 50)

C/Python parity : 100.00%
Accuracy (host) : 93.33%
Accuracy (board) : 93.33%
Host-check (g++) : 100.00% C/Python parity

Latency per inference: mean 212.4 us | p50 208.0 | p95 244.0 | max 260.0
Measured memory: Flash 3412 B (10%) | SRAM 268 B (13%)

RESULT: PASS

```

Field	How to read it
C/Python parity	The number that matters: whether the board agrees with Python sample by sample. Below 100%, the report lists the mismatches (host, board and true label).
Accuracy (host) × (board)	Accuracy against the true label. If both match, the embedded model lost no quality.
Host-check (g++)	Parity of the same C compiled on the PC. If it fails here, the board is not the problem.
Latency	Time per inference (mean, p50, p95 and max), already divided by the repetitions. Use it to size your sampling period.
Measured memory	Real Flash and SRAM reported by the toolchain, next to the pyruleanalyzer estimate.
RESULT	PASS = exit code 0; FAIL = exit code 1.

7. Pitfalls you will run into

- **On AVR, double equals float** (4 bytes). Thresholds are exported with 17 digits and lose precision when parsed by the `atof()` of the board. Samples sitting almost exactly on a threshold may diverge from Python. Parity of 99.x% with every mismatch on a boundary is exactly that — not a logic bug. On the ESP32 (real 8-byte double) the effect disappears.
- **Feature order is a contract.** The board knows no names, only indices. The vector you send must follow the same order used at training time — and `read_features()`, in the production sketch, carries the same obligation.
- **Uno and Nano have 2 KB of SRAM.** Each feature costs 4 bytes in the buffer plus around 14 bytes of text on the serial line. Past ~40 features the Uno gets tight; prefer a Mega or an ESP32.
- **AVR boards reset when the serial port is opened.** The script waits `--boot-delay` seconds (default 2.5 s) and discards the banner before the handshake.
- **ESP32 and the watchdog.** Too many repetitions in a single message can trip it; keep `--reps` at around 200 or below.
- **Timeout.** Large models (a Random Forest with dozens of trees) take milliseconds per inference on an Uno; multiply that by `--reps` before blaming `--timeout`.

8. Common problems

Symptom	Likely cause and fix
<i>The board did not answer the "I" handshake</i>	Wrong port, different baud rate, HIL sketch not flashed, or the IDE Serial Monitor holding the port. Check with <code>arduino-cli board list</code> and close the IDE.

Symptom	Likely cause and fix
arduino-cli not found	Install arduino-cli or pass <code>--arduino-cli C:\path\arduino-cli.exe</code> . To only generate the sketch, use <code>--no-hardware</code> .
pyserial is not installed	<code>pip install pyserial</code> (the package is called pyserial, but the import is serial).
Compilation blows the memory budget	Reduce <code>--max-depth</code> and <code>--n-estimators</code> , or change boards. The estimate already shows up in stage 1.
Parity below 100%	Run with <code>--host-check</code> : if g++ diverges too, the problem is the model/exporter; if only the board diverges, it is almost always float precision on AVR.
Many timeouts	Raise <code>--timeout</code> or lower <code>--reps</code> ; large models are slow on AVR.

9. Using it in continuous integration

With no board, run stage 3 alone — it guarantees the exporter keeps producing valid C that is equivalent to Python:

```
- name: HIL host-check
  run: |
    pip install -e .
    python examples/arduino_hardware_test.py --dataset iris \
      --host-check --no-hardware --min-parity 100
```

On a self-hosted runner with a physical board attached, use the full command with a fixed `--port`. In both cases the exit code already works as the pass criteria: there is no output to parse.

10. Next step: from PC vectors to real sensors

Once the HIL test passes, you know the inference engine is correct on the board. What is left is swapping the data source: in the **production** sketch, fill `read_features()` with your sensor readings, **in the same feature order used at training time**:

```
void read_features(void) {
    features[0] = analogRead(A0) * (5.0 / 1023.0); // e.g. sensor 1 voltage
    features[1] = dht.readTemperature();           // e.g. temperature
    features[2] = dht.readHumidity();               // e.g. humidity
    // ... one line per feature, in training order
}
```

A good practice is to keep both sketches around: the production one for the field, and the HIL one to re-validate the model whenever it is retrained.